# MoveInSync Backend Case Study 2026

**SDE – Java Backend Hiring Assessment · Advanced Track (IIIT & NIT) · Case Study 7**

## Case Study 7: Polygon Zones at Scale – Tariff Zones, No-Go Areas & Trip Attribution

Many rules in employee transport depend on where a cab is: city limits, airport zones with a surcharge, tariff zones billed at different rates, and no-go areas that a cab must never enter at night. These areas are polygons drawn by admins. They can be non-convex, have holes, and contain thousands of vertices. Your job is to answer “which zones is this point in?” for a live stream of GPS pings, split every trip’s distance by zone for billing, and let admins edit zones while the system is running.

### I. Polygon Model

- Validation — reject polygons that are not closed, self-intersect, or have holes outside the outer ring.
- Versioning — every edit creates a new version, and old versions remain queryable.

### II. Point Queries at Speed

- Correct Point-in-Polygon — correct for non-convex shapes and holes, with a defined rule for points exactly on an edge or vertex.
- Spatial Index — with 5,000 polygons and 50,000 pings per second, do not test every polygon for every ping. Use an R-tree, a grid or geohash cells, and explain your choice.

### III. Trip Attribution

- Distance by Zone — for a trip’s path, compute the kilometres inside each tariff zone, splitting segments exactly where they cross zone boundaries.
- Entry and Exit Events — emit zone entry and exit events with dwell time, and raise a no-go alert for a night trip entering a no-go zone.

### IV. Live Edits

- No Downtime — rebuild the index in the background and swap it in atomically, so queries never see a half-built index.
- Trip Consistency — define which polygon version applies to a trip that is in progress when a zone is edited, and apply it consistently.

### Example Use Case

• An airport drop of 42 km is billed as 30.4 km in the city zone and 11.6 km in the airport zone, with the boundary crossing at the toll plaza.

• An admin shrinks a no-go zone at 9:00 PM. Pings after the swap use the new shape, a trip that started at 8:50 PM follows the rule you defined, and query latency does not spike during the rebuild.

### Key Points to Keep in Mind

- Latitude and longitude are not a flat plane. State your projection or distance method and the error it introduces.
- Floating-point comparisons on boundaries need an explicit tolerance. Test points on edges, vertices and inside holes.
- The index swap is a concurrency problem. Show a test that queries continuously during an edit.
- **Prove it:** benchmark point queries per second against 5,000 polygons, and report index build time and memory.

---

## Plus Points in Implementation (Overall Evaluation Criteria)

1. **Authentication:** Implement robust authentication and role-based access (employee vs admin vs operator), with tenant isolation so one client can never read another client’s data.
2. **Cost Estimation – Time and Space:** Analyse the time and space complexity of your core logic, and back it with a small benchmark. State where the system stops scaling and why.

3. **Handling System Failure Cases:** Implement fault-tolerant mechanisms for crashes, timeouts, duplicate and out-of-order messages, and partial failures. Show at least one failure-injection test.
4. **Object-Oriented Programming (OOPS):** Choose a robust OOPS language (Java preferred). Apply encapsulation, inheritance and polymorphism for maintainable, extensible code with clean controller / service / repository layering.
5. **Trade-offs in the System:** Clearly define and document the trade-offs you made, especially consistency vs availability vs latency. Explain what you deliberately left out.
6. **System Monitoring:** Add metrics and structured logging. Expose the counters an on-call engineer would need at 3 AM (lag, error rate, rejections, retries, queue depth).
7. **Caching:** Integrate caching for hot data with a sensible eviction policy. Say exactly what can become stale, for how long, and how staleness is bounded.
8. **Error and Exception Handling:** Build a robust error-handling framework with meaningful, non-leaking error messages. No personal data (phone, email, home location) in logs or error responses.

## What “Advanced Track” Means

- **Proven guarantees** – every hard rule in your case study has an automated test, including at least one concurrency test and one failure test.
- **Measured performance** – a small load test or benchmark with throughput and p50 / p99 latency, reported in the README with the machine you ran it on.
- **Design note** – one or two pages with an architecture diagram, one sequence diagram for the hardest flow, and a failure-mode table (what fails, what the user sees, how the system recovers).
- **One-command run** – the project starts with a single command (for example docker compose up), including any database or message broker it needs.

## Instructions

1. **Read and Understand the Problem Statement:** Carefully read the case study. Understand the requirements, inputs, expected outputs, the hard rules and the scale targets.
2. **Choose a Programming Language:** Java with Spring Boot is preferred. C#, Go, Kotlin or Python are acceptable if you can defend the choice.
3. **Design Your Solution:** Plan first. Decide your data model, the main components, the APIs and the failure modes before writing code. Put this in the design note.
4. **Write the Code:** Write clean, layered code with meaningful names. Keep the core algorithm in plain classes that can be tested without a web server or database.
5. **Test Your Code:** Test normal inputs AND edge cases. Add unit tests for the core logic, plus the concurrency, failure and benchmark tests described in your case study.
6. **Document Your Code:** Add a README with how to run it, your assumptions, the benchmark results, and a short note on the trade-offs you made.
7. **Submit Your Solution:** Push to a public GitHub repository and share the link. Keep a meaningful commit history, not one giant commit.
8. **Demonstration:** Add a short demo video showing the main flows, one failure scenario being handled live, and a few sample API calls (Postman / curl).